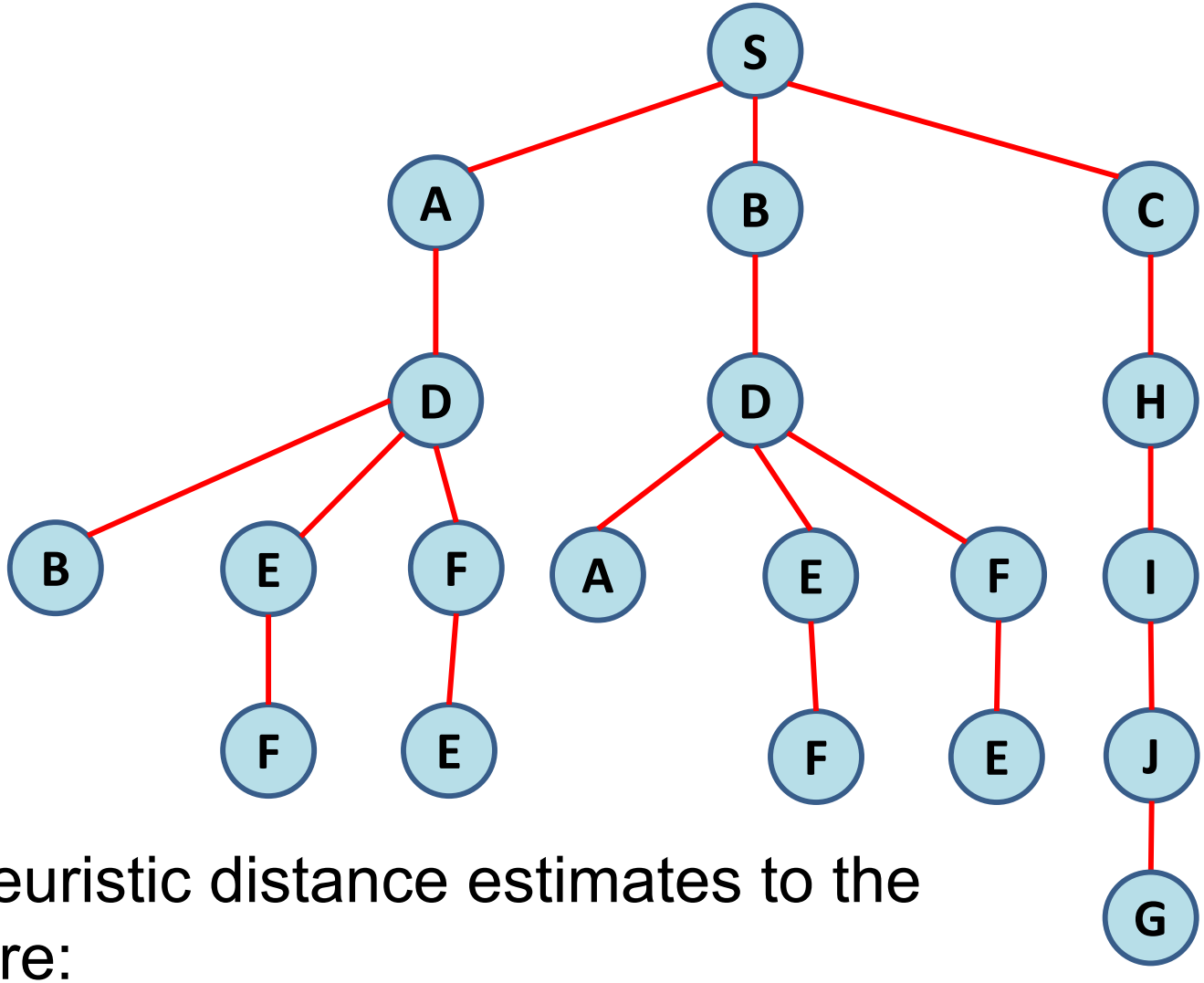
# TUTORIAL#1: SEARCH



## Search Tree




The heuristic distance estimates to the goal are:

S = 10, A = 5, B = 8, C = 6, D = 7, E = 6, F = 5, H = 3, I = 2, J = 1, G = 0

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

Let’s apply the following algorithm to solve the problem:

# Depth-First Search with the Visited List

n.h.reyes@massey.ac.nz

Reference: MIT OpenCourseWare

# SEARCH



## Simple Search Algorithm

**A Search Node is a path from some state X to the start state**
**e.g. (X B A S)**

**The state of a search node is the most recent state of the path**
**e.g. X**

**Let Q be a list of search nodes**
**e.g. (X B A S) (C B A S) …)**

**Let S be the start state.**

Guarantees that each state is reached only once.

1. Initialise Q with search node (S) as only entry; set Visited = (S).
2. If Q is empty, fail. Else, pick some search node N from Q.
3. If state (N) is a goal, return N (we've reached the goal).
4. (Otherwise) Remove N from Q.
5. Find all the descendants of state (N) not in Visited and create all the one-step extensions of N to each descendant.
6. Add the extended paths to Q; add children of state (N) to Visited.
7. Go to Step 2.

**Critical Decisions:**

Step **2**: picking N from Q.
Step **6**: adding extensions of N to Q.

Note: for Any-Path algorithms, the goal may be found at Step 6 already.

**2) DFS:** Pick **first** element of Q; Add path extensions to **front** of Q.

**6) DFS:** Nodes are pulled off the Q and inserted into the Q using a **stack**.

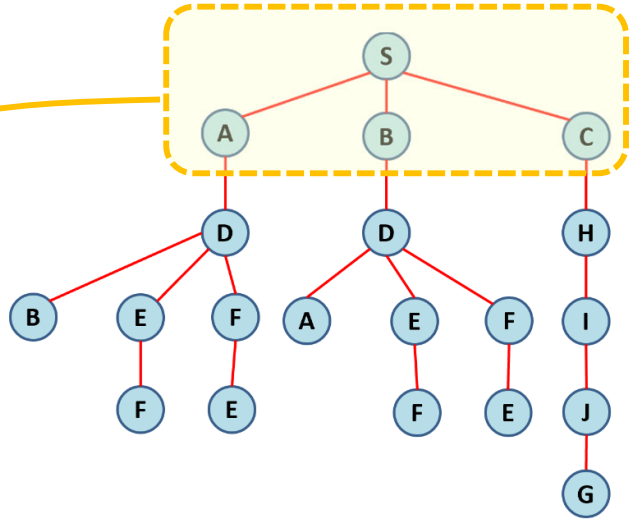
# SEARCH



## Depth-First with Visited List

| Step | Dequeued | Q | Visited |
|---|---|---|---|
| 0 | | (S) | S |
| 1 | (S) | (AS) (BS) (CS) | S,A,B,C |

Search Tree




Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

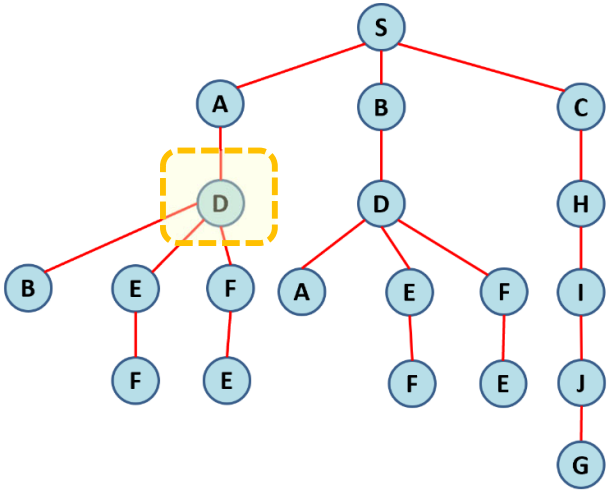
# SEARCH



## Depth-First with Visited List

| Step | Dequeued | Q | Visited |
|---|---|---|---|
| 0 | | (S) | S |
| 1 | (S) | (AS) (BS) (CS) | S,A,B,C |
| 2 | (AS) | (DAS) (BS) (CS) | ", D |

Search Tree




" – means including all previous states visited

Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

# SEARCH

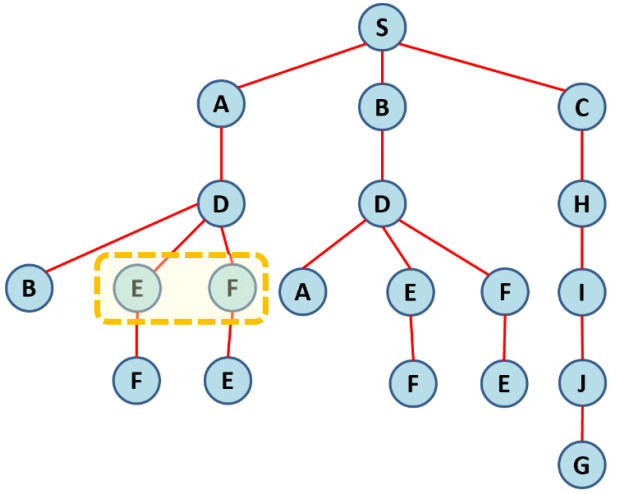


## Depth-First with Visited List

| Step | Dequeued | Q | Visited |
|---|---|---|---|
| 0 | | (S) | S |
| 1 | (S) | (AS) (BS) (CS) | S,A,B,C |
| 2 | (AS) | (DAS) (BS) (CS) | “ D |
| 3 | (DAS) | (EDAS) (FDAS) (BS)(CS) | “, E, F |
| 4 | (EDAS) | (FDAS) (BS) (CS) | “ |
| 5 | (FDAS) | (BS) (CS) | “ |

*continued…*

Search Tree




“ – means including all previous states visited

Not extended to (BDAS) anymore as state B has been visited already

Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

# SEARCH

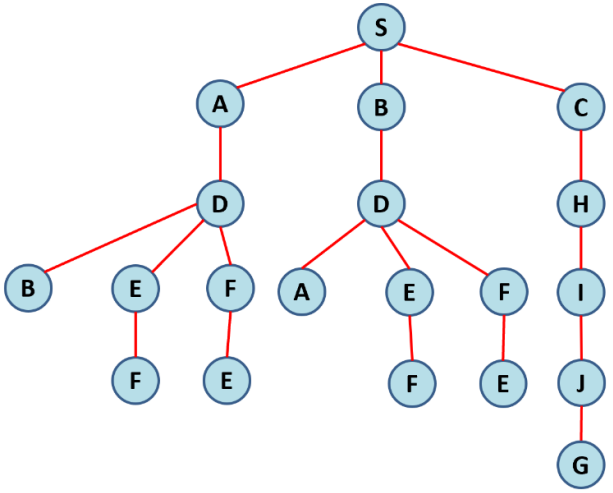


## Depth-First with Visited List

| Step | Dequeued | Q | Visited |
|---|---|---|---|
| 5 | (FDAS) | (BS) (CS) | S,A,B,C,D,E,F |
| 6 | (BS) | (CS) | “ |
| 7 | (CS) | (HCS) | “, H |
| 8 | (HCS) | (IHCS) | “, I |
| 9 | (IHCS) | (JIHCS) | “, J |
| 10 | (JIHCS) | (GJIHCS) | “, G |
| 11 | (GJIHCS) | | “ |

Path = S-C-H-I-J-G

Search Tree




Pick **first** element of Q; Add path extensions to **front** of Q.

In **DFS** – nodes are pulled off the Q and inserted into the Q using a **stack**.

# Depth-First with Visited List

Characterisation:

Sequence of state expansions = ?

Path = ?

Max Q Length = ?

Number of state expansions = ?



# Depth-First with Visited List

## Characterisation:

Sequence of state expansions = S-A-D-E-F-B-C-H-I-J-G

Path = S-C-H-I-J-G

Path length = 5

Max Q Length = 4

Number of state expansions = 11